

VMA

1 介绍

Vulkan Memory Allocator (VMA) 是 AMD 公司提供的的一个 Vulkan 显存管理库，因为 Vulkan 本身是细粒度、较复杂的 API，直接使用 Vulkan 管理显存较麻烦，VMA 封装了 Vulkan 显存管理，使得 Vulkan 开发更简单。

VMA 的优点如下：

- (1) VMA 为创建的 buffer 或 image 选择最佳的显存类型；
- (2) VMA 会分配较大的 VkDeviceMemory，并将创建的 buffer 或 image 放入其中；
- (3) VMA 为创建 buffer 或 image 提供给更简单的接口，用户不需要考虑显存占用、分配 VkDeviceMemory、绑定资源等繁琐的步骤。

2 内存类型分类

2.1 VkMemoryPropertyFlags 属性

2.1.1 *VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT*

特点：

- 可以使用 vkMapMemory() 映射到 CPU 地址空间
- 适合需要频繁 CPU 访问的数据（如 vertex buffer、uniform buffer）
- 对 CPU 可见，可以被映射和直接访问
- 通常性能不如 device local memory

2.1.2 *VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT*

特点:

- 通常位于 GPU 显存中
- GPU 访问速度最快
- 对 CPU 不可见（可以通过 staging buffer 实现可见）

2.1.3 *VK_MEMORY_PROPERTY_HOST_COHERENT_BIT*

特点:

- 写入内存后不需要手动刷新缓存
- 避免了使用 `vkFlushMappedMemoryRanges()` 和 `vkInvalidateMappedMemoryRanges()`

2.1.4 *VK_MEMORY_PROPERTY_HOST_CACHED_BIT*

特点:

- CPU 读取性能更好
- 需要手动刷新缓存
- 适合 CPU 频繁读取但较少写入的场景

2.1.5 *VK_MEMORY_PROPERTY_LAZILY_ALLOCATED_BIT*

特点:

- 延迟内存分配: 允许驱动延迟实际的物理内存分配, 直到真正需要时才进行分配
- 优化 tile-based 渲染: 在移动端, 渲染中使用 on-chip memory 进行中间计算, 只有在 tile 处理完成后才需要将数据写入系统内存。对于某些临时图像 (如 depth buffer), 如果它们的内容不需要在帧之间进行持久化, 就可以使用延迟分配来避免分配实际的物理内存
- 通过减少内存分配和内存带宽使用, 可以提高整体渲染性能

2.2 常见组合

(1) `VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT` |
`VK_MEMORY_PROPERTY_HOST_COHERENT_BIT`
用于 CPU 和 GPU 之间频繁数据传输

(2) `VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT` |
`VK_MEMORY_PROPERTY_HOST_CACHED_BIT`
常见于从 GPU 读取数据到 CPU 的场景

(3) `VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT` |
`VK_MEMORY_PROPERTY_HOST_COHERENT_BIT` |
`VK_MEMORY_PROPERTY_HOST_CACHED_BIT`
适用于需要频繁双向数据传输的场景

(4) `VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT` |
`VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT`
常见于统一内存架构 (UMA)

3 内存映射

3.1 介绍

内存映射 (Memory Mapping) 用于 CPU 内存和显存之间的读写, 对应的 flag 必须有 `VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT`, 通过调用 `vkMapMemory()` 和 `vkUnmapMemory()` 实现.

但是 VMA 不建议直接使用这两个接口, 因为:

- 对同一个 `VkDeviceMemory` 进行多次 mapping 是不合法的
- 在 Vulkan 内部 mapping 不是引用计数的
- 操作不是线程安全的

VMA 建议在 `VmaAllocationCreateInfo::flags` 中必须指定以下两个 flag 其中之一:

- *VMA_ALLOCATION_CREATE_HOST_ACCESS_SEQUENTIAL_WRITE_BIT*
- *VMA_ALLOCATION_CREATE_HOST_ACCESS_RANDOM_BIT*

3.2 Copy 函数

VMA 使用 `vmaCopyMemoryToAllocation()` 和 `vmaCopyAllocationToMemory()` 完成内存和显存之间的拷贝。

3.3 Mapping 函数

VMA 使用 `vmaMapMemory()` 和 `vmaUnmapMemory()` 完成对 `VulkanDeviceMemory` 的映射和取消映射。VMA 在内部实现了引用计数，支持同一个 `VulkanDeviceMemory` 同时进行多次 map，vulkan memory 会在第一次 mapping 完成 map，在最后一次 unmapping 完成 unmap。此处需注意 `vmaMapMemory()` 和 `vmaUnmapMemory` 的调用次数必须相同

3.4 持续 map

Vulkan 支持 `VkDeviceMemory` 保持 mapped 状态，VMA 支持 `VmaAllocationCreateInfo::flags` 设置 *VMA_ALLOCATION_CREATE_MAPPED_BIT*，VMA 会自动将 `VkDeviceMemory` 保持 mapped 状态，可以随时通过 CPU 指针访问 `VkDeviceMemory`，不必调用 map 或 unmap 操作。

4 显存预算

4.1 查询显存

VMA 通过 `vmaGetHeapBudgets()` 查询当前显存使用量以及可用显存量。相比于 `vmaCalculateStatistics()`，`vmaGetHeapBudgets()` 返回信息不同且速度更快。一般程序运行时使用 `vmaGetHeapBudgets()` 查询显存使用量，debug 阶段使用 `vmaCalculateStatistics()` 查询显存使用量。

更建议使用拓展 *VK_EXT_memory_budget*，当该拓展开启时，VMA 会自动使用该拓展查询显存，对显存的当前使用量和可用量的查询结果更加准确。

4.2 控制显存用量

如果当前申请的显存超出了可用显存的量，Vulkan 可能会成功或失败，行为是未知的，有风险的。VMA 建议在申请显存时设置 `VMA_ALLOCATION_CREATE_WITHIN`。当申请的显存大小超出了可用的显存量时，VMA 会不进行显存申请。

5 资源别名

因为 Vulkan 和 Direct3D 12 支持手动进行显存管理，我们可以在同一块显存定义多个资源别名，支持资源覆盖。这可以用于节省显存，但是使用需要掌握整个渲染流程，对资源的生命周期进行管理。如果 texture 或者 buffer 在使用过程中没有重叠，可以绑定到同一块显存，即使 width、height、format 等参数都不同。

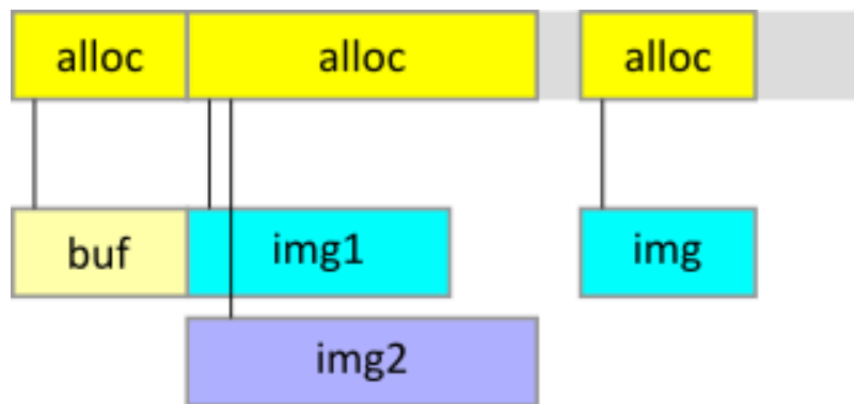


图 1: Resource Alias

VMA 支持这样的操作，但是需要计算以下参数：

- allocation size = max(size of each image)
- allocation alignment = max(alignment of each image)
- allocation memoryTypeBits = bitwise AND(memoryTypeBits of each image)

代码实例：

```

1  // A 512x512 texture to be sampled.
2  VkImageCreateInfo img1CreateInfo = { VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO };
3  img1CreateInfo.imageType = VK_IMAGE_TYPE_2D;
4  img1CreateInfo.extent.width = 512;
5  img1CreateInfo.extent.height = 512;
6  img1CreateInfo.extent.depth = 1;
7  img1CreateInfo.mipLevels = 10;
8  img1CreateInfo.arrayLayers = 1;
9  img1CreateInfo.format = VK_FORMAT_R8G8B8A8_SRGB;
10 img1CreateInfo.tiling = VK_IMAGE_TILING_OPTIMAL;
11 img1CreateInfo.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;
12 img1CreateInfo.usage = VK_IMAGE_USAGE_TRANSFER_DST_BIT | VK_IMAGE_USAGE_SAMPLED_BIT;
13 img1CreateInfo.samples = VK_SAMPLE_COUNT_1_BIT;
14
15 // A full screen texture to be used as color attachment.
16 VkImageCreateInfo img2CreateInfo = { VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO };
17 img2CreateInfo.imageType = VK_IMAGE_TYPE_2D;
18 img2CreateInfo.extent.width = 1920;
19 img2CreateInfo.extent.height = 1080;
20 img2CreateInfo.extent.depth = 1;
21 img2CreateInfo.mipLevels = 1;
22 img2CreateInfo.arrayLayers = 1;
23 img2CreateInfo.format = VK_FORMAT_R8G8B8A8_UNORM;
24 img2CreateInfo.tiling = VK_IMAGE_TILING_OPTIMAL;
25 img2CreateInfo.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;
26 img2CreateInfo.usage = VK_IMAGE_USAGE_SAMPLED_BIT | VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT;
27 img2CreateInfo.samples = VK_SAMPLE_COUNT_1_BIT;
28
29 VkImage img1;
30 res = vkCreateImage(device, &img1CreateInfo, nullptr, &img1);
31 VkImage img2;
32 res = vkCreateImage(device, &img2CreateInfo, nullptr, &img2);
33
34 VkMemoryRequirements img1MemReq;
35 vkGetImageMemoryRequirements(device, img1, &img1MemReq);
36 VkMemoryRequirements img2MemReq;
37 vkGetImageMemoryRequirements(device, img2, &img2MemReq);
38
39 VkMemoryRequirements finalMemReq = {};

```

```

40 finalMemReq.size = std::max(img1MemReq.size, img2MemReq.size);
41 finalMemReq.alignment = std::max(img1MemReq.alignment, img2MemReq.alignment);
42 finalMemReq.memoryTypeBits = img1MemReq.memoryTypeBits & img2MemReq.memoryTypeBits;
43 // Validate if (finalMemReq.memoryTypeBits != 0)
44
45 VmaAllocationCreateInfo allocCreateInfo = {};
46 allocCreateInfo.preferredFlags = VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT;
47
48 VmaAllocation alloc;
49 res = vmaAllocateMemory(allocator, &finalMemReq, &allocCreateInfo, &alloc, nullptr);
50
51 res = vmaBindImageMemory(allocator, alloc, img1);
52 res = vmaBindImageMemory(allocator, alloc, img2);
53
54 // You can use img1, img2 here, but not at the same time!
55
56 vmaFreeMemory(allocator, alloc);
57 vkDestroyImage(allocator, img2, nullptr);
58 vkDestroyImage(allocator, img1, nullptr);

```

显存复用需要添加恰当的同步操作，确保 *img1* 和 *img2* 在使用的时间内没有重叠。VMA 提供 `vmaCreateAliasingBuffer()`, `vmaCreateAliasingBuffer2()`, `vmaCreateAliasingImage()`, `vmaCreateAliasingImage2()` 来支持 resource aliasing。其中带有版本号 2 表示支持额外参数 `allocationLocalOffset`。

6 自定义显存池

6.1 介绍

VMA 提供默认的显存池实现，支持对各种显存类型进行管理，显存池的大小会自动增长，分配的内存块大小是可变的且是自动管理的，只需调用 `VmaAllocationCreateInfo::pool = null`。

当有以下需求时，可以考虑自定义显存池：

- 需要设置特定格式的 allocation
- 强制使用固定大小的显存块

- 限制显存池的最大容量
- 提前分配最小容量或固定容量的显存块
- 在分配的内存块子集进行 defragmentation（碎片复用）

调用过程分为以下三步：

- (1) 创建 `VmaPoolCreateInfo`
- (2) 调用 `vmaCreatePool()` 来创建显存池对象 `VmaPool`
- (3) 在创建 allocation 时通过 `VmaAllocationCreateInfo::pool` 指定显存池对象

6.2 选择 memory type index

当创建显存池时,需要显式设置 memory type index。可以通过 `vmaFindMemoryTypeIndexForBufferInfo()` 和 `vmaFindMemoryTypeIndexForImageInfo()` 来获取 buffer 或 image 的 memory type index。

6.3 何时不使用自定义显存池

用自定义显存池的劣势：

- 每个显存池有自己的 `VkDeviceMemory` 块的集合，但是其中可能有部分或全部是空的，这会造成大量的显存浪费
- 必须手动设置 memory type index。对于默认的显存池，VMA 会自动选择最合适的 memory type index
- 对于自定义的显存池，对于特定显存类型的分配操作如果失败，会直接返回失败；默认的显存池，会尝试兼容的显存类型
- 如果设置 `VmaPoolCreateInfo::blockSize != 0`，那么每个显存块有相同的大小；而默认的显存池会先尝试分配较小的显存块，如果不满足显存需求，才会分配更大的显存块

在以下情况，默认显存池可能会比自定义显存池更优：

- 希望分配固定大小或者生命周期的显存块
- 希望分配的 buffer 和 image 相互隔离（VMA 自动实现）

- 希望已经 map 和未 map 的显存块相互隔离 (VMA 自动实现)
- 希望为默认的显存块设置自定义的大小, 可以通过 `VmaAllocatorCreateInfo::preferredLargeHeapBlockSize` 来设置
- 希望为 allocation 设置自定义的 memory type, 可以将 `VmaAllocationCreateInfo::memoryTypeBits` 设置为 $(1U \ll \text{myMemoryTypeIndex})$
- 希望为创建的 buffer 设置最小的 alignment, 可以用默认的显存池来调用 `vmaCreateBufferWithAlignment()`

6.4 线性分配算法

VMA 支持线性分配策略, 通过在创建 `VmaPool` 对象时, 在 `VmaPoolCreateInfo::flags` 添加 `VMA_POOL_CREATE_LINEAR_ALGORITHM_BIT`. 如图2所示, VMA 不会使用之前的显存块的空闲空间, 而是从新的显存块开始分配。



图 2: Linear Allocation

对于自定义的显存池, 其内部存在多种分配机制, 如 `free-at-once`, `stack`, `double stack`, 以及 `ring buffer`。

6.4.1 Free-at-once

如图3所示, 当释放所有 allocation, 显存池变为空, 显存分配会从头开始。在释放完所有 allocation 后, 这一机制能加速新的 allocation 的创建。

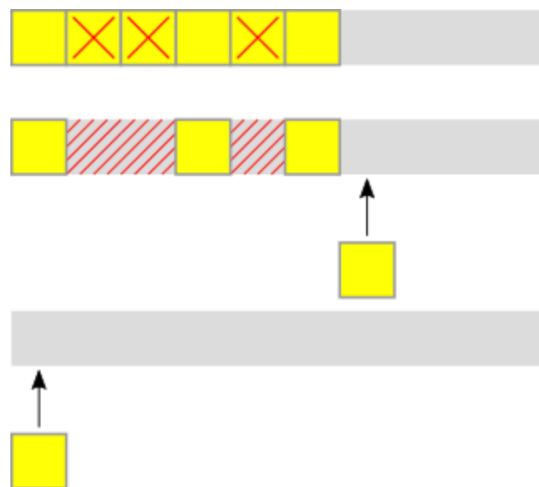


图 3: Free-at-once

6.4.2 Stack

当释放最新创建的 allocation，该 allocation 的空间可以复用。如图4所示，实现类似栈的功能。

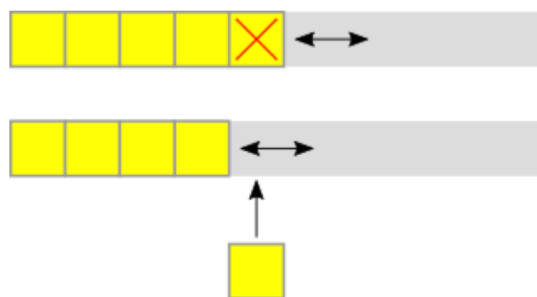


图 4: Stack

6.4.3 Double stack

如图5所示，对于双栈机制：

- 第一个栈作为默认的栈，从 offset 0 开始分配

- 第二个栈，从末端向低 offset 进行分配

当需要从第二个栈进行分配,需要将 `VMA_ALLOCATION_CREATE_UPPER_ADDRESS_BIT` 添加到 `VmaAllocationCreateInfo::flags`

Double stack 只支持一个显存块,必须将 `VmaPoolCreateInfo::maxBlockCount` 设置为 1。

当两个栈相遇,没有足够的空间分配新的 allocation,会返回 `VK_ERROR_OUT_OF_DEVICE_MEMORY`。



图 5: Double Stack

6.4.4 Ring buffer

如图6所示,会形成环形结构,如果在某一端没有足够的空间分配,会从另一端开始分配。

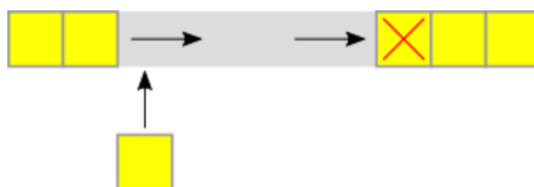


图 6: Ring Buffer

注意:

当创建自定义显存池时,如果设置了 `VMA_POOL_CREATE_LINEAR_ALGORITHM_BIT`,那么不支持去碎片化 (Defragmentation)。

7 去碎片化

7.1 介绍

间断性地申请和释放不同大小的显存，随着时间的推进，会导致找不到连续的显存来进行分配，即使空余的显存足够大。

VMA 引入去碎片化（Defragmentation）功能，用于缓解这个问题。

7.2 流程

7.2.1 调用 `vmaBeginDefragmentationPass()`

- 返回要被释放的 allocation 列表，这是一个耗时的操作（defragmentation 与显存池绑定）。
- 创建临时的 allocation，用于存储拷贝数据。

7.2.2 对 allocation 列表进行遍历

- 获取要释放的 allocation 的 info
- 创建新的 buffer 或 image，绑定临时的 allocation
- 进行数据拷贝
- 释放原本的 image 或 buffer

7.2.3 调用 `vmaEndDefragmentationPass()`

- 释放被删除的 allocation 对应的显存
- 将原本的 `VmaAllocation` 对象指向新的显存
- 释放空的 `VkDeviceMemory` 块

8 参考

VMA Github

VMA Documentation